

# Konspekt projektu badawczego

## Zastosowanie Monte Carlo Tree Search (MCTS/UCT) do gry w warcaby $10 \times 10$ bez obowiązkowego bicia z damkami bijącymi po całej diagonalu

Jakub Kindracki

Wiktor Kobielski

11 maja 2026

## 1 Opis problemu

Przedmiotem projektu jest zastosowanie algorytmu Monte Carlo Tree Search (MCTS), w szczególności jego wariantu Upper Confidence Bound applied to Trees (UCT), do stworzenia sztucznej inteligencji grającej w wybraną grę dwuosobową z pełną informacją. Grą wybraną do badań są warcaby na planszy  $10 \times 10$  z modyfikacjami klasycznych zasad: *bicie nie jest obowiązkowe* oraz *damki biją wzdłuż całej diagonalu*, co odpowiada zasadom zbliżonym do warcabów międzynarodowych w aspekcie ruchu damki, lecz różnym w aspekcie obowiązku bicia.

Brak obowiązkowego bicia istotnie zwiększa współczynnik rozgałęzienia drzewa gry — w każdej pozycji, w której istnieje bicie, gracz może je wykonać lub wybrać dowolny inny ruch. Oznacza to, że MCTS musi rozważać znacznie szerszy zbiór akcji niż w klasycznych warcabach, a heurystyki ukierunkowane na bicia (typowe dla silników warcabowych) mają ograniczone zastosowanie. Latające damki natomiast wzmacniają wartość promocji i wprowadzają silne, dalekosiężne interakcje pomiędzy figurami, co może wpływać na jakość losowych symulacji. Ta kombinacja zasad czyni grę nietrywialnym przypadkiem do badania algorytmu MCTS i jego rozszerzeń.

Stan gry reprezentujemy jako planszę  $10 \times 10$  z 20 pionkami po każdej stronie, ustawionymi na ciemnych polach pierwszych czterech rzędów każdego z graczy. Gra kończy się zwycięstwem strony, której przeciwnik nie ma legalnego ruchu (brak figur lub całkowite zablokowanie). W celu zapewnienia, aby eksperymenty kończyły się w rozsądnym czasie, wprowadzimy limit liczby pólruchów bez bicia i bez ruchu pionka — przekroczenie limitu skutkuje remisem.

## 2 Wstępny przegląd literatury

Algorytm MCTS w postaci UCT został zaproponowany przez Kocsisa i Szepesváriego [1]. W niniejszym projekcie opieramy się przede wszystkim na przeglądzie Świechowskiego, Godlewskiego, Sawickiego i Mańdziuka [2], który systematyzuje modyfikacje MCTS publikowane w ostatniej dekadzie i grupuje je dla gier z pełną informacją w trzy kategorie: *action reduction* (ograniczanie zbioru akcji), *UCT alternatives* (modyfikacje reguły selekcji) oraz *early termination* (skręcanie fazy symulacji). Z tego podziału wybieramy dwie modyfikacje, po jednej z pierwszych dwóch kategorii, jako najlepiej dopasowane do specyfiki naszej gry (duży współczynnik rozgałęzienia, dostępna prosta funkcja oceny pozycji, deterministyczna gra dwuosobowa z pełną informacją):

- **RAVE / AMAF** (Rapid Action Value Estimation, All Moves As First) – z kategorii *UCT alternatives* [2], oryginalnie zaproponowane przez Gelly’ego i Silvera [3, 4]. Wynik symulacji jest przypisywany nie tylko ruchowi faktycznie wykonanemu w węźle, lecz wszystkim ruchom o tym samym kluczu, które wystąpiły w dalszej części ployoutu. Statystyki AMAF są łączone z klasycznymi statystykami UCT za pomocą malejącej wagi  $\beta(n)$ , co przyspiesza zbieżność wartości akcji w rzadko odwiedzanych węzłach – efekt szczególnie cenny w grach o dużym współczynniku rozgałęzienia.

- **Progressive Bias** – z kategorii *action reduction* [2], oryginalnie wprowadzona przez Chasłota i in. [5]. Reguła wyboru węzła jest wzmocniona członem heurystycznym  $w \cdot h(\text{stan})/(n_i + 1)$ , gdzie  $h$  jest statyczną funkcją oceny pozycji, a  $n_i$  liczbą wizyt dziecka. Wpływ członu maleje wraz ze wzrostem  $n_i$ , dzięki czemu heurystyka kieruje przeszukiwaniem we wczesnej fazie, a w miarę gromadzenia danych decyduje statystyka Monte Carlo.

Z kategorii *early termination* [2] świadomie rezygnujemy – jej zastosowanie wymagałoby silniejszej funkcji oceny niż ta, którą dysponujemy, a w naszym wariancie zasad losowe playouty do końca gry pozostają wykonalne czasowo dzięki limitowi półruchów bez bicia.

Łącznie porównamy zatem trzy warianty MCTS: **czysty UCT**, **UCT+RAVE** oraz **UCT+progressive bias**. Wszystkie dzielą tę samą strukturę pętli (selekcja – ekspansja – losowa symulacja do końca gry – propagacja wsteczna) oraz ten sam mechanizm zrównoleglenia (root parallelization: niezależne drzewa w procesach roboczych z agregacją liczników wizyt na końcu).

W kontekście samej gry warcaby  $10 \times 10$  bez obowiązku bicia z latającymi damkami pozostają wariantem stosunkowo słabo zbadanym pod kątem MCTS. Przegląd [2] wielokrotnie wskazuje, że w grach o wysokim współczynniku rozgałęzienia modyfikacje wykorzystujące wiedzę domenową lub współdzielenie statystyk akcji zwykle wyraźnie wyprzedzają podstawowy UCT.

**Gracz heurystyczny.** Bazowy gracz heurystyczny działa w prostym schemacie *greedy jednopoziomowym*: dla każdego legalnego ruchu z bieżącej pozycji wykonuje go na kopii stanu, oblicza wartość statycznej funkcji oceny powstałej pozycji i wybiera ruch maksymalizujący ocenę (z deterministycznym rozstrzygnięciem remisów dzięki ziarnu RNG). Funkcja oceny jest liniową kombinacją:

- **materiału** – pionek o wartości 1,0, damka 3,0;
- **zaawansowania pionka** – mały bonus za bliskość do rzędu promocji (premiuje rozwój i potencjał uzyskania damki);
- ocena zwracana jest w postaci znormalizowanej biały/(biały+czarny)  $\in [0, 1]$ , dzięki czemu działa w obu kolorach po prostym odwróceniu znaku.

Świadomie zachowujemy heurystykę prostą – ma być realistycznym, lecz słabym przeciwnikiem porównawczym i jednocześnie tym samym członem  $h(\text{stan})$  używanym w wariancie progressive bias, co zapewnia spójność porównań.

### 3 Hipotezy badawcze

Zgodnie z wymaganiami projektu stawiamy cztery hipotezy: trzy wymagane (porównania graczy) oraz jedną autorską, związaną bezpośrednio z modyfikacją zasad badanego wariantu warcabów.

#### H1 (UCT vs heurystyka).

Czysty UCT z wystarczającym limitem iteracji ( $\geq 10\,000$  na ruch) gra istotnie skuteczniej od gracza czysto heurystycznego, osiągając  $> 60\%$  wygranych w meczu z naprzemienną zmianą kolorów.

#### H2 (RAVE poprawia UCT).

UCT z rozszerzeniem RAVE wygrywa  $> 60\%$  partii z czystym UCT przy tym samym limicie iteracji.

#### H3 (Progressive bias poprawia UCT).

UCT z heurystycznym progressive bias wygrywa  $> 60\%$  partii z czystym UCT, a jego przewaga jest największa przy małym limicie iteracji ( $\leq 2\,000$ /ruch), gdy losowe symulacje nie wystarczają do oceny pozycji. Wraz ze wzrostem liczby iteracji oczekujemy zaniku przewagi.

#### H4 (krawędzie vs środek planszy).

W badanym wariancie warcabów heurystyki premiujące figury przy krawędzi planszy są skuteczniejsze niż heurystyki premiujące figury w centrum. Hipotezę zweryfikujemy porównując dwa warianty statycznej funkcji oceny  $h$  – jeden z dodatnim bonusem za zbliżenie do krawędzi bocznych, drugi z dodatnim bonusem za zbliżenie do centrum – wstawione do obu graczy wykorzystujących heurystykę (gracza czysto heurystycznego oraz UCT + progressive bias). Mecze rozegramy w dwóch konfiguracjach dla każdego rodzaju gracza: heurystyka-krawędzie vs heurystyka-centrum. Hipotezę uznamy za potwierdzoną, jeśli w obu konfiguracjach wariant „krawędzie” osiąga  $> 50\%$  wygranych.

## 4 Sposób weryfikacji hipotez

### 4.1 Plan eksperymentów

Każdą hipotezę zweryfikujemy poprzez zorganizowanie turniejów *round-robin* pomiędzy graczami przy ustalonym budżecie iteracji / czasu. Dla każdej pary graczy rozegramy  $N \geq 50$  partii (po  $N/2$  z każdym kolorem), aby wyeliminować wpływ pierwszego ruchu.

**Zbiory danych.** Ponieważ jest to gra dwuosobowa, "danymi" są wyniki rozegranych partii. Dla zapewnienia powtarzalności:

- Każdy gracz przyjmuje parametr **seed**; ziarna pochodne wyprowadzamy deterministycznie z ziarna bazowego.
- Każda konfiguracja eksperymentu (para graczy, budżet, wariant zasad) zostanie powtórzona z  $\geq 5$  różnymi ziarnami bazowymi, co pozwoli oszacować rozrzut wyników.
- Wszystkie partie zostaną zapisane (sekwencja ruchów + ziarno), umożliwiając powtórzenie konkretnego przebiegu.

**Mierzone wielkości.** Procent wygranych, remisów i przegranych z 95%-przedziałem ufności (Wilson lub bootstrap), średnia długość partii, średni czas na ruch, średnia liczba odwiedzonych węzłów drzewa.

**Testy istotności.** Dla porównań dwóch graczy zastosujemy dwustronny test proporcji (lub bootstrap przedziału ufności na różnicy procentów wygranych). Hipotezę uznajemy za potwierdzoną, jeśli  $p < 0,05$  i efekt jest spójny dla różnych ziaren.

**Plan opracowania wyników.** Wyniki przedstawimy w postaci: wykresów słupkowych z przedziałami ufności (porównania graczy), wykresów krzywych (skuteczność vs budżet iteracji), tabel zbiorczych w raporcie. Surowe dane zostaną dołączone jako pliki JSON/CSV.

### 4.2 Testy z udziałem ludzi

Zaimplementujemy interfejs przeglądarkowy (Flask + HTML/JS) umożliwiający grę człowiek-komputer. Przeprowadzimy testy z co najmniej 5 osobami (różny poziom znajomości warcabów). Każda osoba rozegra po jednej partii z każdym wariantem AI; zbierzemy ankietę dotyczącą subiektywnej oceny "ludzkiego" charakteru i siły gry, a także wyniki obiektywne.

## 5 Harmonogram działań

| Termin    | Zadanie                                                                                                                                    |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Tydzień 1 | Implementacja silnika gry (warcaby $10 \times 10$ z wybranymi zasadami), testy jednostkowe generatora ruchów, gracz losowy i heurystyczny. |
| Tydzień 2 | Implementacja czystego UCT oraz wariantów RAVE i progressive bias; root parallelization (multiprocessing).                                 |
| Tydzień 3 | Interfejs przeglądarkowy (Flask) i runner eksperymentalny CLI z obsługą ziarna oraz raportowania (JSON/CSV).                               |
| Tydzień 4 | Eksperymenty H1–H3 (porównania graczy AI) oraz H4 (porównanie wariantów zasad); testy z udziałem 5+ osób.                                  |
| Tydzień 5 | Opracowanie wyników, pisanie raportu, przygotowanie prezentacji.                                                                           |

## 6 Ogólny projekt techniczny

Całość projektu zostanie napisana w języku **Python 3** (cross-platform, jeden ekosystem dla silnika gry, MCTS, runnera eksperymentów oraz interfejsu webowego). Korzystamy wyłącznie ze standardowej biblioteki oraz kilku popularnych pakietów:

- `multiprocessing` (stdlib) – root parallelization MCTS (niezależne drzewa w procesach roboczych, agregacja liczników wizyt na końcu).
- `Flask` – lekki serwer webowy obsługujący interfejs gry człowiek–komputer (REST + HTML/JS).
- `matplotlib` / `pandas` – analiza wyników i wykresy do raportu końcowego.

**Reprodukowalność.** Wszyscy gracze przyjmują **seed**. Eksperymenty alternują kolory i używają ziaren wyprowadzonych deterministycznie z bazowego. Pełna konfiguracja partii (gracze, ziarno, zasady) zapisywana jest w wynikach, co pozwala odtworzyć dowolny przebieg.

## Literatura

- [1] L. Kocsis and C. Szepesvári, *Bandit based Monte-Carlo Planning*, ECML, 2006.
- [2] M. Świechowski, K. Godlewski, B. Sawicki, J. Mańdziuk, *Monte Carlo Tree Search: a review of recent modifications and applications*, Artificial Intelligence Review, 56:2497–2562, 2023.
- [3] S. Gelly and D. Silver, *Combining online and offline knowledge in UCT*, ICML, 2007.
- [4] S. Gelly and D. Silver, *Monte-Carlo tree search and rapid action value estimation in computer Go*, Artificial Intelligence, 175(11):1856–1875, 2011.
- [5] G. M. J.-B. Chaslot, M. H. M. Winands, H. J. van den Herik, J. W. H. M. Uiterwijk, B. Bouzy, *Progressive strategies for Monte-Carlo tree search*, New Mathematics and Natural Computation, 4(3):343–357, 2008.